

Supabase Myticket DB

1. TABLES STRUCTURE

Table: profiles (User Management)

```
CREATE TABLE profiles (  
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  username VARCHAR(255),  
  full_name VARCHAR(255),  
  role VARCHAR(50) DEFAULT 'support', -- 'admin' or 'support'  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

Table: tickets (Main Tickets Table)

```
CREATE TABLE tickets (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ticket_id VARCHAR(20) UNIQUE NOT NULL, -- Format: #0001  
  ticket_type VARCHAR(100) NOT NULL, -- "Issue", "Request", "Bug", etc.  
  service_type VARCHAR(100) NOT NULL, -- "SMS", "Bot", "F2M", etc.  
  domain VARCHAR(255), -- Optional, required for certain service types  
  priority VARCHAR(20) DEFAULT 'Medium', -- 'High', 'Medium', 'Low'  
  status VARCHAR(20) DEFAULT 'Waiting', -- 'Waiting', 'Done'  
  description TEXT NOT NULL,  
  solution TEXT,  
  creator VARCHAR(255) NOT NULL, -- Email of creator  
  assigned_to VARCHAR(255), -- Email of assignee (NULL if unassigned)  
  created_at TIMESTAMP DEFAULT NOW(),
```

```
updated_at TIMESTAMP DEFAULT NOW(),  
deleted_at TIMESTAMP, -- Soft delete support  
deleted_by UUID REFERENCES profiles(id)  
);
```

Table: ticket_attachments (JPG Attachments)

```
CREATE TABLE ticket_attachments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ticket_id UUID NOT NULL REFERENCES tickets(id) ON DELETE CASCADE,  
  file_name VARCHAR(255) NOT NULL,  
  file_url TEXT NOT NULL, -- Supabase Storage URL  
  file_size INTEGER,  
  uploaded_by VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Table: ticket_activity_log (Audit Trail)

```
CREATE TABLE ticket_activity_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ticket_id UUID NOT NULL REFERENCES tickets(id) ON DELETE CASCADE,  
  action VARCHAR(100) NOT NULL, -- 'created', 'updated', 'deleted', 'assigned',  
  'status_changed'  
  changed_by VARCHAR(255) NOT NULL,  
  old_value TEXT,  
  new_value TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

2. INDEXES FOR PERFORMANCE

```
CREATE INDEX idx_tickets_status ON tickets(status);
```

```
CREATE INDEX idx_tickets_creator ON tickets(creator);
CREATE INDEX idx_tickets_assigned_to ON tickets(assigned_to);
CREATE INDEX idx_tickets_priority ON tickets(priority);
CREATE INDEX idx_tickets_created_at ON tickets(created_at DESC);
CREATE INDEX idx_tickets_service_type ON tickets(service_type);
CREATE INDEX idx_ticket_attachments_ticket_id ON
ticket_attachments(ticket_id);
CREATE INDEX idx_activity_log_ticket_id ON ticket_activity_log(ticket_id);
```

3. ROW LEVEL SECURITY (RLS) POLICIES

profiles Table Policies

-- Enable RLS

```
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
```

-- Users can view all profiles

```
CREATE POLICY "Users can view all profiles"
```

```
ON profiles FOR SELECT
```

```
USING (true);
```

-- Only admins can update user roles

```
CREATE POLICY "Admins can update profiles"
```

```
ON profiles FOR UPDATE
```

```
USING (
```

```
    auth.uid() IN (
```

```
        SELECT id FROM profiles WHERE role = 'admin'
```

```
    )
```

```
)
```

```
WITH CHECK (
```

```
auth.uid() IN (  
    SELECT id FROM profiles WHERE role = 'admin'  
)  
);
```

tickets Table Policies

-- Enable RLS

```
ALTER TABLE tickets ENABLE ROW LEVEL SECURITY;
```

-- Support staff can view all tickets

```
CREATE POLICY "Support can view all tickets"
```

```
ON tickets FOR SELECT
```

```
USING (  
    auth.uid() IN (  
        SELECT id FROM profiles WHERE role IN ('admin', 'support')  
    )  
    AND deleted_at IS NULL  
);
```

-- Support staff can create tickets

```
CREATE POLICY "Support can create tickets"
```

```
ON tickets FOR INSERT
```

```
WITH CHECK (  
    auth.uid() IN (  
        SELECT id FROM profiles WHERE role IN ('admin', 'support')  
    )  
);
```

-- Support staff can update their own tickets

```
CREATE POLICY "Support can update tickets"
```

```
ON tickets FOR UPDATE
```

```
USING (
```

```
    auth.uid() IN (
```

```
        SELECT id FROM profiles WHERE role IN ('admin', 'support')
```

```
    )
```

```
    AND deleted_at IS NULL
```

```
)
```

```
WITH CHECK (
```

```
    auth.uid() IN (
```

```
        SELECT id FROM profiles WHERE role IN ('admin', 'support')
```

```
    )
```

```
);
```

```
-- Only admins can delete tickets (soft delete via updated_at & deleted_at)
```

```
CREATE POLICY "Only admins can delete tickets"
```

```
ON tickets FOR DELETE
```

```
USING (
```

```
    auth.uid() IN (
```

```
        SELECT id FROM profiles WHERE role = 'admin'
```

```
    )
```

```
);
```

```
ticket_attachments Table Policies
```

```
-- Enable RLS
```

```
ALTER TABLE ticket_attachments ENABLE ROW LEVEL SECURITY;
```

```
-- Support staff can view attachments
```

```
CREATE POLICY "Support can view attachments"
ON ticket_attachments FOR SELECT
USING (
    auth.uid() IN (
        SELECT id FROM profiles WHERE role IN ('admin', 'support')
    )
);
```

-- Support staff can upload attachments

```
CREATE POLICY "Support can upload attachments"
ON ticket_attachments FOR INSERT
WITH CHECK (
    auth.uid() IN (
        SELECT id FROM profiles WHERE role IN ('admin', 'support')
    )
);
```

-- Only admins can delete attachments

```
CREATE POLICY "Only admins can delete attachments"
ON ticket_attachments FOR DELETE
USING (
    auth.uid() IN (
        SELECT id FROM profiles WHERE role = 'admin'
    )
);
```

ticket_activity_log Table Policies

-- Enable RLS

```
ALTER TABLE ticket_activity_log ENABLE ROW LEVEL SECURITY;
```

```
-- Support staff can view activity logs
```

```
CREATE POLICY "Support can view activity logs"
```

```
ON ticket_activity_log FOR SELECT
```

```
USING (
```

```
    auth.uid() IN (
```

```
        SELECT id FROM profiles WHERE role IN ('admin', 'support')
```

```
    )
```

```
);
```

```
-- System can insert logs
```

```
CREATE POLICY "System can insert logs"
```

```
ON ticket_activity_log FOR INSERT
```

```
WITH CHECK (true);
```

4. USER PERMISSIONS MATRIX

Action	Support Staff	Admin
View all tickets		
View my tickets		

Action	Support Staff	Admin
View unassigned tickets	✓	✓
Create ticket	✓	✓
Edit ticket (status, assignee)	✓	✓
Edit ticket description	✓	✓
Assign/unassign ticket	✓	✓
Delete ticket	✗	✓
View audit logs	✓	✓
Upload attachment	✓	✓
Delete attachment	✗	✓
Manage user roles	✗	✓

5. SUPABASE STORAGE BUCKET

-- Create bucket for ticket attachments

CREATE BUCKET ticket_attachments;

-- Set bucket to private

UPDATE buckets

SET public = false

WHERE name = 'ticket_attachments';

Storage Policies:

-- Support staff can upload to their folder

CREATE POLICY "Support can upload ticket attachments"

ON storage.objects FOR INSERT

WITH CHECK (

bucket_id = 'ticket_attachments'

AND auth.role() = 'authenticated'

);

-- Support staff can view attachments

CREATE POLICY "Support can view attachments"

ON storage.objects FOR SELECT

USING (bucket_id = 'ticket_attachments');

-- Only admins can delete files

CREATE POLICY "Admins can delete attachments"

ON storage.objects FOR DELETE

USING (

bucket_id = 'ticket_attachments'

AND auth.uid() IN (

SELECT id FROM profiles WHERE role = 'admin'

)

);

6. HELPER FUNCTIONS

Function: Auto-generate Ticket ID

```
CREATE OR REPLACE FUNCTION generate_ticket_id()
RETURNS VARCHAR AS $$
DECLARE
    next_num INTEGER;
BEGIN
    SELECT COUNT(*) + 1 INTO next_num FROM tickets;
    RETURN '#' || LPAD(next_num::text, 4, '0');
END;
$$ LANGUAGE plpgsql;
```

-- Trigger to auto-set ticket_id

```
CREATE OR REPLACE FUNCTION set_ticket_id()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.ticket_id IS NULL THEN
        NEW.ticket_id := generate_ticket_id();
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_set_ticket_id
BEFORE INSERT ON tickets
FOR EACH ROW
EXECUTE FUNCTION set_ticket_id();
```

Function: Log Activity

```
CREATE OR REPLACE FUNCTION log_ticket_activity()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF TG_OP = 'INSERT' THEN  
        INSERT INTO ticket_activity_log (ticket_id, action,  
changed_by)  
        VALUES (NEW.id, 'created', NEW.creator);  
    ELSIF TG_OP = 'UPDATE' THEN  
        INSERT INTO ticket_activity_log (ticket_id, action,  
old_value, new_value, changed_by)  
        VALUES (NEW.id, 'updated', ROW(OLD.*)::text,  
ROW(NEW.*)::text, NEW.creator);  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_log_activity  
AFTER INSERT OR UPDATE ON tickets  
FOR EACH ROW  
EXECUTE FUNCTION log_ticket_activity();
```

7. DATABASE VIEWS FOR QUERIES

View: Ticket Statistics

```

CREATE VIEW ticket_stats AS
SELECT
    COUNT(*) as all_tickets,
    COUNT(CASE WHEN status = 'Waiting' THEN 1 END) as waiting,
    COUNT(CASE WHEN assigned_to IS NULL THEN 1 END) as unassigned,
    COUNT(CASE WHEN status = 'Done' THEN 1 END) as done
FROM tickets
WHERE deleted_at IS NULL;

```

View: My Tickets (Parameterized)

```

CREATE VIEW my_tickets AS
SELECT *
FROM tickets
WHERE deleted_at IS NULL
AND creator = current_user_email();

```

8. QUICK REFERENCE: TAB/SCOPE MAPPING

Tab Name	Query Filter	SQL WHERE Clause
All Tickets	scope=all	deleted_at IS NULL
My Tickets	scope=my	creator = current_user AND deleted_at IS NULL
Unassigned	scope=unassigned	assigned_to IS NULL AND deleted_at IS NULL

9. INITIALIZATION SQL (Copy & Paste)

-- 1. Create profiles table

```
CREATE TABLE IF NOT EXISTS profiles (  
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE  
  CASCADE,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  username VARCHAR(255),  
  full_name VARCHAR(255),  
  role VARCHAR(50) DEFAULT 'support',  
  created_at TIMESTAMP DEFAULT NOW(),
```

```
updated_at TIMESTAMP DEFAULT NOW()  
);
```

-- 2. Create tickets table

```
CREATE TABLE IF NOT EXISTS tickets (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ticket_id VARCHAR(20) UNIQUE NOT NULL,  
  ticket_type VARCHAR(100) NOT NULL,  
  service_type VARCHAR(100) NOT NULL,  
  domain VARCHAR(255),  
  priority VARCHAR(20) DEFAULT 'Medium',  
  status VARCHAR(20) DEFAULT 'Waiting',  
  description TEXT NOT NULL,  
  solution TEXT,  
  creator VARCHAR(255) NOT NULL,  
  assigned_to VARCHAR(255),  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  deleted_at TIMESTAMP,  
  deleted_by UUID REFERENCES profiles(id)  
);
```

-- 3. Create attachments table

```
CREATE TABLE IF NOT EXISTS ticket_attachments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ticket_id UUID NOT NULL REFERENCES tickets(id) ON DELETE  
  CASCADE,
```

```
file_name VARCHAR(255) NOT NULL,  
file_url TEXT NOT NULL,  
file_size INTEGER,  
uploaded_by VARCHAR(255) NOT NULL,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

-- 4. Create activity log table

```
CREATE TABLE IF NOT EXISTS ticket_activity_log (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    ticket_id UUID NOT NULL REFERENCES tickets(id) ON DELETE  
    CASCADE,  
    action VARCHAR(100) NOT NULL,  
    changed_by VARCHAR(255) NOT NULL,  
    old_value TEXT,  
    new_value TEXT,  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

-- 5. Enable RLS on all tables

```
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;  
ALTER TABLE tickets ENABLE ROW LEVEL SECURITY;  
ALTER TABLE ticket_attachments ENABLE ROW LEVEL SECURITY;  
ALTER TABLE ticket_activity_log ENABLE ROW LEVEL SECURITY;
```

-- 6. Create indexes

```
CREATE INDEX idx_tickets_status ON tickets(status);
```

```
CREATE INDEX idx_tickets_creator ON tickets(creator);  
CREATE INDEX idx_tickets_assigned_to ON tickets(assigned_to);  
CREATE INDEX idx_tickets_priority ON tickets(priority);  
CREATE INDEX idx_tickets_created_at ON tickets(created_at DESC);  
CREATE INDEX idx_tickets_service_type ON tickets(service_type);  
CREATE INDEX idx_ticket_attachments_ticket_id ON  
ticket_attachments(ticket_id);  
CREATE INDEX idx_activity_log_ticket_id ON ticket_activity_log(ticket_id);
```

**### Ready to copy and paste into Supabase SQL Editor! All tables,
permissions, and admin deletion capabilities are included.###**